

Problem 1:

Given a sequence of integers $a_1, a_2, \dots, a_n$. Find a contiguous subarray of exactly k consecutive elements such that the GCD of those k elements are maximized.

Input:

- Line 1: n, k ($2 \leq k \leq n \leq 10^3$)
- Next line: $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$).

Output:

- The maximum GCD of k consecutive elements.

Example:

INPUT	OUTPUT
10 3 2 6 4 3 18 12 24 8 7 5	6

Solution : Problem 1: Max GCD of Subarray Size K

Pattern: Sliding Window / Range Queries

Constraints: $n \leq 10^3$ (Target $O(N^2)$ minimum, $O(N \log N)$ or $O(N)$ for full marks)++

- **Baseline (Brute Force):**
 - **Logic:** Loop through every window of size K . Compute GCD for all K elements from scratch.
 - **Time:** $O(N * K * \log(\max A))$. (N windows, K elements per window, $O(\log(\max A))$ for Euclidean algorithm).
 - **Space:** $O(1)$.
 - **Bottleneck:** Recalculating the GCD of overlapping elements in adjacent windows.
- **Optimal Solution 1: Sparse Table**

- **Logic:** GCD is idempotent ($\text{GCD}(x,x) = x$). Precompute a Sparse Table to query overlapping intervals.
- **Time:** Precomputation takes $O(N \log N * \log(\max A))$. Queries take $O(1)$ table lookups plus $O(\log(\max A))$ for the math. Total Time: $O(N \log N * \log(\max A))$.
- **Space:** $O(N \log N)$ to store the table.
- **Optimal Solution 2: Two-Stack Queue (Best)**
 - **Logic:** Implement a sliding window using a queue built from two stacks. The stacks maintain the running GCD. Adding/removing elements is amortized $O(1)$.
 - **Time:** Strictly $O(N * \log(\max A))$.
 - **Space:** $O(K)$ to store the window in the stacks.
- **Oral Exam "Gotchas":**
 - **For a sparse table solution,** we construct a table $\text{st}[i][j] = \text{gcd}$ of interval starting at i with length of 2^j . For example $\text{st}[0][2] = \text{gcd}(\text{elements from } 0 \text{ to } (2^{2-1} = 3))$. Since $\text{gcd}(x,x) = x$ (idempotent) so we do not need to care about overlapping intervals. We first construct the table and then query, To answer a query on range $[L,R]$, let $\text{len} = R-L+1$ and choose the largest j such that $2^j \leq \text{len}$. Then compute:
 $\text{gcd}(\text{st}[L][j], \text{st}[R-2^j+1][j])$ and do this for all k -size window
 - **Why not use a standard queue?** You cannot "undo" or reverse a GCD operation when the left-most element slides out of the window. You must use a data structure that supports range queries or the two-stack method.
 - **Worst-case input for GCD:** Consecutive Fibonacci numbers (maximizes steps in the Euclidean algorithm).

Problem 2:

Let $g(n)$ be the number of pairs (p, q) such that $p + q = 2 \times n$ and p, q are prime. Two pairs are considered different if one of the two numbers belongs to one pair but not the other. Given integer n . Compute the value of the expression: $f(n) = g(2) + g(3) + \dots + g(n)$.

Input:

- Contains an integer n ($3 \leq n \leq 10^3$).

Output:

- A single integer — the value of $f(n)$.

Example:

INPUT	OUTPUT
9	12

Problem 2: Sum of Prime Pairs (Goldbach's Variation)

Pattern: Number Theory / Sieve of Eratosthenes / Precomputation

Constraints: $n \leq 10^3$ (Max value to check is $2n = 2000$. $O(N^2)$ time is completely fine here).

- **Baseline (Brute Force):**
 - **Logic:** For every integer i from 2 to n , loop a variable P from 2 up to i . Check if P is prime and if $(2 \times i - P)$ is prime using a standard trial division function (checking up to the square root).
 - **Time:** $O(N^2 * \sqrt{N})$. (N outer loops, N inner loops, $\sqrt{N}$ for primality test).
 - **Space:** $O(1)$.
 - **Bottleneck:** Redundant primality checks. You are calculating whether the same numbers are prime hundreds of times.
- **Optimal Solution (Sieve of Eratosthenes):**

- **Logic:**
 1. Precompute all primes up to $2*n$ (2000) using a boolean Sieve array. This gives $O(1)$ primality lookups.
 2. Keep a running sum. Loop i from 2 to n .
 3. Inside, loop P through all primes $\leq i$. If `isPrime[2*i - P]` is true, increment your sum.
- **Time:** Sieve takes $O(M \log \log M)$ where $M = 2*n$. The nested loop takes $O(N * (\text{Primes} \leq N))$. Total Time is roughly $O(N^2 / \log N)$, which is blindingly fast for $N=1000$.
- **Space:** $O(N)$ to store the boolean Sieve array of size $2*N$.
- **Oral Exam "Gotchas" & Follow-ups:**
 - **How do you avoid double-counting pairs?** (e.g., counting (3,5) and (5,3) as two different pairs). You solve this by strictly looping P from 2 only up to i (which is exactly half of $2*i$). This forces $P \leq Q$, ensuring each unique pair is only seen once.
 - **What if the lecturer asks to handle 10^5 queries instead of just one?** You would pivot to a Prefix Sum approach. You precompute an array where `ans[i]` stores the number of pairs for $2*i$, and then build a prefix sum array. Then, any query for fn can be answered in $O(1)$ time!
 - **The Ultimate Flex:** If the lecturer asks "What if $N = 10^5$?", the $O(N^2)$ pair-matching loop will Time Out. You can mention that counting how many ways two primes can sum to every even number up to $2*N$ in $O(N \log N)$ time requires advanced math called the Fast Fourier Transform (FFT) applied to polynomials. (You don't need to know how to code it, just knowing the name of the optimization is an instant A).
 - **Note:** $n = 5$ so $2*n = 10 = \text{target}$ then $g(5) = 2$ for $3+7 = 10$ and $5+5 = 10$. To avoid counting duplicates we strictly set for $p \leq n$. Then if `prime(p)` and if `prime(target-p)` then we increment $g(n)$ and sum over $g(n)$ to yield $f(n)$.

Problem 3:

Given two integer sequences $A = (a_1, a_2, \dots, a_m)$ and $B = (b_1, b_2, \dots, b_n)$, find an element a_i in sequence A and an element b_j in sequence B such that $|a_i + b_j|$ is minimized ($1 \leq i \leq m; 1 \leq j \leq n$).

Input:

- Line 1 contains two positive integers $m, n \leq 10^5$
- Line 2 contains m integers $a_1, a_2, \dots, a_m$ (for all i : $|a_i| < 2 \times 10^9$)
- Line 3 contains n integers $b_1, b_2, \dots, b_n$ (for all j : $|b_j| < 2 \times 10^9$)

Output: Output the two indices i and j of the corresponding elements.

Example:

INPUT	OUTPUT
4 5 1 8 2 9 -5 -6 3 -7 -4	2 4

Explanation: $|a_2 + b_4| = |8 + (-7)| = 1$

Problem 3: Minimize Absolute Sum of Two Elements

Pattern: Sorting + Two Pointers (or Binary Search)

Constraints: $m, n \leq 10^5$ (Target $O(N \log N)$). Values up to 2×10^9 (Requires 64-bit integers!).

- **Baseline (Brute Force):**
 - **Logic:** Nested loops. For every element in A, check every element in B. Calculate the absolute sum and update the minimum.
 - **Time:** $O(N * M)$. This will Time Out because $10^5 * 10^5 = 10^{10}$ operations.
 - **Space:** $O(1)$.

- **Bottleneck:** Blindly checking every pair. If $A[i]$ is a huge positive number, we don't need to test it against huge positive numbers in B; we only care about large negative numbers in B.
- **Optimal Solution: Sorting + Two Pointers**
 - **Logic:**
 1. Store the arrays as pairs of (value, original_index) so we do not lose the indices when we sort.
 2. Sort array A in ascending order. Sort array B in ascending order.
 3. Place a pointer i at the start of A (smallest values) and a pointer j at the end of B (largest values).
 4. Calculate $current_sum = A[i].value + B[j].value$. Update your minimum absolute sum if it's smaller.
 5. If $current_sum < 0$: The sum is too negative. We need a bigger number to get closer to 0, so move i forward ($i++$).
 6. If $current_sum > 0$: The sum is too positive. We need a smaller number, so move j backward ($j--$).
 7. If $current_sum == 0$: We found the absolute best answer. Break early.
 - **Time:** $O(M \log M + N \log N)$ for sorting. The two-pointer traversal takes strictly $O(M + N)$. Total Time: $O(N \log N)$.
 - **Space:** $O(M + N)$ to store the pairs.
- **Oral Exam "Gotchas":**
 - **Integer Overflow (The biggest trap!):** The maximum value is $2 * 10^9$. The sum of two max values is $4 * 10^9$, which exceeds the limit of a standard 32-bit signed int. You MUST use long long for the sum and minimum variables, or your code will fail on hidden test cases.
 - **Losing the Answer:** If you just call `sort(A)` and `sort(B)`, you lose the original positions needed for the output. Bundling the value with the index before sorting is mandatory.
 - **1-Based Indexing:** The problem asks for 1-based indices in the output. If your arrays are 0-indexed, make sure to add +1 when storing the original indices.

Problem 4:

Given n and a sequence of n numbers $a_1, a_2, \dots, a_n$. An inversion is defined as a pair of elements a_i and a_j in the sequence satisfying $i < j$ and $a_i > a_j$.

Task: Count the number of inversions in the sequence.

Input:

- The first line contains an integer n ($1 \leq n \leq 5 \times 10^4$).
- The second line contains n integers $a_1, a_2, \dots, a_n$ ($-2^{31} \leq a_i \leq 2^{31}$).

Output:

- A single integer — the number of inversions in the sequence.

Example:

INPUT	OUTPUT
6 1 2 4 3 5 1	5

Problem 4: Count Inversions

Pattern: Divide and Conquer (Merge Sort Variation) / Fenwick Tree

Constraints: $N \leq 5 * 10^4$. (An $O(N^2)$ brute force will require $2.5 * 10^9$ operations and will Time Out. You must use an $O(N \log N)$ approach).

- **Baseline (Brute Force):**
 - **Logic:** Use two nested loops. The outer loop 'i' goes from 0 to N-1. The inner loop 'j' goes from i+1 to N-1. If $A[i] > A[j]$, increment the inversion count.
 - **Time:** $O(N^2)$.
 - **Space:** $O(1)$.
 - **Bottleneck:** We are repeatedly comparing elements that could be processed in groups.
- **Optimal Solution 1: Merge Sort (The Standard A-Grade Answer)**

- **Logic:** Modify the standard Merge Sort algorithm. When merging a "Left" half and a "Right" half (both already sorted), we compare Left[i] and Right[j].
 - If Left[i] ≤ Right[j]: No inversion.
 - If Left[i] > Right[j]: We found an inversion! Because the Left half is sorted, if Left[i] is greater than Right[j], then *every* remaining element in the Left half is also greater than Right[j]. We immediately add (mid - i + 1) to our inversion count and move j forward.
- **Time:** strictly $O(N \log N)$, identical to standard Merge Sort.
- **Space:** $O(N)$ for the temporary array used during the merge step.
- **Optimal Solution 2: Fenwick Tree / Binary Indexed Tree (The Flex)**
 - **Logic:** You can use a Fenwick tree to keep track of how many numbers you have seen so far that are smaller than the current number.
 - **Time:** $O(N \log N)$.
 - **Space:** $O(N)$.
 - **Note:** Because the input values can be negative and up to 2^{31} , you cannot use the values directly as array indices. You would have to apply "Coordinate Compression" first to map the huge values to ranks (1, 2, 3...).
- **Oral Exam "Gotchas":**
 - **Integer Overflow for the Answer:** The maximum number of inversions happens when the array is sorted in descending order. The formula is $N * (N - 1) / 2$. For $N = 50,000$, this is 1,249,975,000. This barely fits inside a standard 32-bit signed integer (limit $2.14 * 10^9$). However, if the lecturer asks "What if $N = 10^5$ ", the max inversions becomes $\sim 5 * 10^9$, which will overflow a standard int. **Always use long long for the inversion count variable!**
 - **Array Modification:** Merge sort alters the original array. If the lecturer asks, "What if the original array must remain untouched for later use?", tell them you would simply pass a copy of the array into your function, not the original reference.

Problem 5:

Given string A and string B consisting only of lowercase letters. String B is said to appear at position i of string A if:

$$A_i = B_1, A_{i+1} = B_2, \dots, A_{i+\text{len}(B)-1} = B_{\text{len}(B)}$$

Task: Find all positions where string B appears in A .

Input

- Line 1: string A .
- Line 2: string B .

The lengths of A , B are at most 10^6 .

Output

Print all found positions on one line in increasing order. If B does not appear in A leave the output empty.

Example:

INPUT	OUTPUT
aaaaa aa	1 2 3 4

Problem 5: Substring Search (Find all occurrences)

Pattern: String Matching (KMP Algorithm / Rolling Hash)

Constraints: Lengths up to 10^6 . (An $O(N * M)$ Brute Force will result in 10^{12} operations and will Time Out. You must use an $O(N + M)$ algorithm).

- **Baseline (Brute Force):**
 - **Logic:** Use two pointers. For every starting index i in string A , check if the next characters match string B character by character. If a mismatch occurs, reset the check starting at $i + 1$.

- **Time:** $O(N * M)$ in the worst case (e.g., $A = \text{"aaaaaaa..."}$, $B = \text{"aaaaab"}$).
- **Space:** $O(1)$.
- **Bottleneck:** When a mismatch occurs after checking many identical characters, the brute force approach completely forgets the matches it just saw and starts over.
- **Optimal Solution 1: Knuth-Morris-Pratt (KMP) Algorithm (Standard A-Grade Answer)**
 - **Logic:** KMP avoids redundant checks. It precomputes an LPS array (Longest Prefix Suffix) for string B. The LPS array tells the algorithm: "If a mismatch happens here, we already know the previous characters matched, so don't start from scratch. Just shift the pattern by a specific amount."
 - **Time:** $O(M)$ to build the LPS array + $O(N)$ to search. Total Time: $O(N + M)$.
 - **Space:** $O(M)$ to store the LPS array.
- **Optimal Solution 2: Rabin-Karp Algorithm (Rolling Hash) (Alternative)**
 - **Logic:** Convert string B into a mathematical hash number. Then, slide a window of size M across string A, updating the hash of the window in $O(1)$ time. If the hash of the window matches the hash of B, check if the actual characters match (to prevent hash collisions).
 - **Time:** Average $O(N + M)$.
 - **Space:** $O(1)$.
- **Oral Exam "Gotchas":**
 - **Overlapping Matches:** Look closely at the example ($A = \text{"aaaaa"}$, $B = \text{"aa"}$, $\text{Output} = \text{"1 2 3 4"}$). This proves matches can overlap! When you find a full match in KMP, you cannot just reset your pattern pointer j to 0. You must set $j = \text{lps}[j - 1]$ to continue checking for overlaps.
 - **1-Based Indexing:** Standard strings in C++ are 0-indexed. When you print the final positions, you must add 1 to the calculated index.

Problem 6:

Given k sorted sequences of integers (each sequence is non-decreasing). Elements may repeat.

Task: Merge all k sequences into a single non-decreasing sorted list.

Input

- The first line contains integer k — the number of sequences. ($1 \leq k \leq 10^4$)
- k The next k lines each contain one sorted sequence; each line has at most 500 integers.
- The total number of elements across all k sequences does not exceed 10^4 .

Output

A single line containing all elements merged in non-decreasing order.

Example:

INPUT	OUTPUT
3 1 4 5 1 3 4 2 6	1 1 2 3 4 4 5 6

Problem 6: Merge K Sorted Sequences

Pattern: Priority Queue (Min-Heap) / K-Way Merge

Constraints: $K \leq 10^4$, Total Elements (N) $\leq 10^4$.

- **Baseline (Brute Force):**
 - **Logic:** Read every single number from all K sequences and push them into one giant 1D array. Call the standard `sort()` function on the giant array.
 - **Time:** $O(N \log N)$, where N is the total number of elements across all lists.
 - **Space:** $O(N)$ to store the giant array.

- **Bottleneck:** Calling a standard sort completely ignores the fact that the individual sequences are already sorted! We are doing unnecessary work.
- **Optimal Solution: Min-Heap (Priority Queue)**
 - **Logic:**
 1. Create a Min-Heap.
 2. Look at the very first element of all K sequences. Push these K elements into the heap. (You must store the value, which array it came from, and its index in that array).
 3. Pop the smallest element from the heap and print it (or add to result array).
 4. Look at which array that popped element came from. Grab the *next* element from that specific array and push it into the heap.
 5. Repeat until the heap is empty.
 - **Time:** $O(N \log K)$. The heap never holds more than K elements. Inserting/extracting takes $O(\log K)$ time. We do this for all N elements.
 - **Space:** $O(K)$ for the heap. (Significantly better memory usage than brute force if N is massive and we stream the output!).
- **Alternative Optimal: Divide and Conquer**
 - **Logic:** Merge arrays in pairs using the standard 2-array merge logic (like in Merge Sort). Merge array 1 with 2, array 3 with 4, etc. Then merge the results.
 - **Time:** $O(N \log K)$.
- **Oral Exam "Gotchas" & Implementation Traps:**
 - **Input Parsing (The C++ Trap):** The problem states each line has "at most 500 integers," meaning we do not know the exact length of each sequence beforehand! You cannot just use `cin >> num` in a simple loop. You MUST read the input line-by-line using `getline(cin, line)` and then extract the numbers from that string using `stringstream`.

- **Custom Comparator:** A standard `priority_queue` in C++ is a Max-Heap. To make it a Min-Heap that stores custom structs, you must overload the `>` (greater than) operator.
- **Empty Arrays:** Always check if a sequence is completely empty before trying to push its first element into the heap.

Problem 7:

Given a sequence of n non-negative integers representing the elevation map of a terrain. After heavy rain, water is trapped (see illustration below). Water flows to adjacent lower positions if the current level exceeds the neighbor. Both ends of the terrain border the sea at level 0.



Task: Compute the total amount of water trapped after the rain.

Input

- The first line contains a positive integer n — the width of the terrain. ($0 \leq n \leq 3 \times 10^4$).
- The second line contains n non-negative integers h_i — the elevation at position i of the terrain. ($0 \leq h_i \leq 10^5$)

Output

A single integer — the total amount of trapped water.

Example:

INPUT	OUTPUT
12 0 1 0 2 1 0 1 3 2 1 2 1	6

Problem 7: Trapping Rain Water

Pattern: Two Pointers / Array Precomputation (Dynamic Programming)

Constraints: $N \leq 3 \cdot 10^4$, Heights $\leq 10^5$.

- **Baseline (Brute Force):**

- **Logic:** For every single block at index i , water can only be trapped above it if there are taller blocks somewhere to its left AND somewhere to its right. The water level above block i is exactly: $\min(\text{tallest_bar_on_left}, \text{tallest_bar_on_right}) - \text{height}[i]$.
- **Time:** $O(N^2)$. For every block, you scan the entire left side and the entire right side.
- **Space:** $O(1)$.
- **Bottleneck:** We are repeatedly scanning the same left and right sides to find maximums we should already know.

- **Optimal Solution: Two Pointers (Best - $O(1)$ Space)**

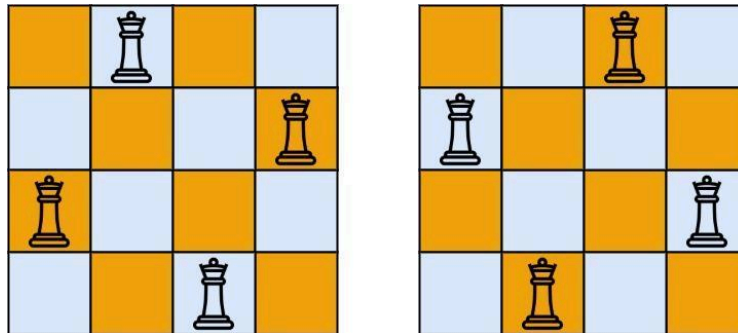
- **Logic:** Place a **left** pointer at index 0 and a **right** pointer at index $n-1$. Keep track of **max_left** and **max_right**.
 - If $\text{height}[\text{left}] < \text{height}[\text{right}]$: We know that the right side is a secure "wall". The water level at the **left** pointer depends entirely on **max_left**. If $\text{height}[\text{left}] \geq \text{max_left}$, we update our wall. Otherwise, we trap water: $\text{total_water} += (\text{max_left} - \text{height}[\text{left}])$. Then move **left** inward.
 - If $\text{height}[\text{right}] \leq \text{height}[\text{left}]$: The left side is our secure wall. We do the exact same logic but using **max_right** and moving the **right** pointer inward.
- **Time:** $O(N)$ (A single pass).
- **Space:** strictly $O(1)$.

- **Oral Exam "Gotchas":**

- **Integer Overflow on the Sum:** This is the trap! The width is up to 30,000. The height is up to 100,000. If the terrain is shaped like a giant "V", the trapped water could be roughly $30,000 * 100,000 = 3,000,000,000$. A standard 32-bit signed `int` maxes out at 2,147,483,647. Your `total_water` variable **MUST** be a `long long`, or you will fail the hidden large test cases.
- **Empty Terrain:** What if $N=0$ or $N=1$ or $N=2$? You cannot trap water with fewer than 3 blocks. Be sure your code checks if $(n < 3)$ and returns 0 immediately to prevent out-of-bounds errors.

Problem 8:

Given a chessboard of size $n \times n$. Find all ways to place n queens on the board such that no two queens attack each other (queens move along rows, columns, and diagonals). Below are 2 example placements of 4 queens on a 4×4 .



Input

- Contains integer n — the board side length and number of queens.
($1 \leq n \leq 9$)

Output

- The first line contains integer m — the number of distinct valid placements.
- The following lines contain matrices representing the board configurations, separated by blank lines.
- Each matrix has n rows. Each row contains n characters:
 - Character '.' represents an empty cell

- o Character 'Q' represents a cell containing a queen

Example:

INPUT	OUTPUT
4	2 .Q.. ...Q Q.. ..Q. ..Q. Q.. ...Q .Q..

Problem 8: N-Queens

Pattern: Backtracking (Depth-First Search)

Constraints: $N \leq 9$. (An $O(N!)$ factorial time complexity is expected and will easily pass since $9!$ is only 362,880 maximum operations).

- **Baseline / Optimal Solution:**

- o **Logic:** Place queens one row at a time. In the current row, try placing a queen in each column from left to right. Before placing, check if the cell is attacked. If it's safe, place the queen and recursively move to the next row. If you hit a dead end, "backtrack" by removing the queen and trying the next column.
- o **The Optimization Trick:** Do NOT use nested loops to check if a spot is attacked! Use three boolean arrays to achieve $O(1)$ lookups:
 1. `col[c]`: Is there a queen in column `c`?
 2. `diag1[r - c + n]`: Is the major diagonal (top-left to bottom-right) taken? (Math trick: the difference of row and col is constant on this diagonal. We add `n` to prevent negative array indices).

3. `diag2[r + c]`: Is the minor diagonal (top-right to bottom-left) taken? (Math trick: the sum of row and col is constant on this diagonal).
- **Time:** $O(N!)$. The first row has N choices, the second has roughly $N-1$, etc.
- **Space:** $O(N)$ for the boolean arrays and recursion stack, plus memory to store the answers.
- **Oral Exam "Gotchas":**
 - **Output Ordering:** The problem requires you to print the *total number of solutions first* (the integer `m`), and then print the boards. This means you cannot just print the boards as you find them. You **MUST** store all valid boards in a list (`vector<vector<string>>`), then print the size of the list, and finally loop through and print the boards.
 - **Array Sizes:** Remember that while there are N columns, a grid has $2 * N - 1$ diagonals. Your boolean arrays for `diag1` and `diag2` must be sized to at least $2 * n$.

Problem 9:

Given two strings s and p consisting of lowercase Latin letters 'a'–'z'. In pattern string p there may also be the special characters '*' and '.' with the following meanings:

- The character '*' means the preceding character may be repeated zero or more times. For example, "abbbb" can be matched by "ab*", and "a" can also be matched by "ab*".
- The character '.' matches exactly one arbitrary character. For example, "abc" can be matched by "a.c" or "ab.".

Task: Determine whether string s can be fully matched by pattern p or not.

Input

- The first line contains string s . ($\text{len}(s) \leq 20$).
- The second line contains pattern p . ($\text{len}(p) \leq 30$).

Output

- Output “true” if string s can be fully matched by pattern p , and “false” otherwise.

Example:

INPUT	OUTPUT
mississippi mis*is*p*.	false
aab c*a*b	true

Problem 9: Regular Expression Matching

Pattern: 2D Dynamic Programming (DP)

Constraints: lengths ≤ 30 . (This means an $O(N * M)$ DP solution is perfectly optimal. Recursion without memoization will Time Out because it checks overlapping subproblems exponentially).

- **Baseline (Brute Force / Pure Recursion):**
 - **Logic:** Compare the first characters. If the next character in the pattern is a `*`, branch into two possibilities: either ignore the `*` and its preceding character (0 occurrences), or keep the `*` and move the string pointer forward if the characters match (1 or more occurrences).
 - **Time:** $O(2^{(N+M)})$ in the worst case due to massive branching.
 - **Space:** $O(N+M)$ for the recursion stack.
 - **Bottleneck:** We solve the same substring matches over and over again.
- **Optimal Solution: 2D Dynamic Programming (DP Table)**
 - **Logic:** Create a 2D boolean grid `dp[i][j]`. This represents "Does the first i characters of the string match the first j characters of the pattern?"
 - **The Rules:**

1. If the current characters match (or pattern has `.`): $dp[i][j] = dp[i-1][j-1]$ (Inherit the match status from the previous characters).
 2. If the pattern has `*`: We have two choices.
 - Choice A (Zero occurrences): Look two steps back in the pattern. $dp[i][j] = dp[i][j-2]$.
 - Choice B (One or more occurrences): If the character before the `*` matches the current string character, we inherit from the row above. $dp[i][j] = dp[i-1][j]$.
 - We use an OR statement: $dp[i][j] = \text{Choice A} \ || \ \text{Choice B}$.
- **Time:** $O(N * M)$. We fill out a grid of size $N \times M$.
 - **Space:** $O(N * M)$ to store the boolean grid.
 - **Oral Exam "Gotchas" & Follow-ups:**
 - **The Base Cases (Crucial!):** The examiner will ask: "What if the string is empty but the pattern is `a*b*`?" An empty string *can* match a pattern if the pattern consists of zero-occurrences! Your code **MUST** pre-fill the first row ($i=0$) to handle patterns that can reduce to an empty string.
 - **1-Based Indexing:** The DP table should be size $[n+1][m+1]$. Index 0 represents an "empty string/pattern". This prevents negative index crashes when checking $i-1$ or $j-1$. Because the table is 1-based, looking at the actual string characters requires $s[i-1]$ and $p[j-1]$.
 - **Memory Optimization Flex:** If the lecturer asks to improve the $O(N * M)$ space, tell them: "Because the current row i only ever looks at the current row i and the previous row $i-1$, we don't need the whole grid. We can optimize the space to $O(M)$ using just two 1D arrays."

Problem 10:

Given a binary matrix of size $m \times n$ where each cell contains 0 or 1.

Task: Find the largest square submatrix containing only 0s or only 1s.

Example illustration:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input

- The first line contains two positive integers m and n — the number of rows and columns, respectively. ($1 \leq m, n \leq 100$).
- m The next m lines each contain n space-separated 0s and 1s.

Output

- The side length of the largest square containing only 0s or only 1s.

Example:

INPUT	OUTPUT
4 5 1 0 1 0 0 1 0 1 1 1 1 1 1 1 1 1 0 0 1 0	2

Problem 10: Largest Uniform Square Submatrix

Pattern: 2D Dynamic Programming (DP)

Constraints: $m, n \leq 100$. (An $O(N * M)$ DP solution is optimal. A brute force approach would take $O((N*M)^2)$ which is extremely risky and shows a lack of optimization knowledge).

- **Baseline (Brute Force):**
 - **Logic:** For every single cell (i, j) , treat it as the top-left corner of a square. Expand a square of size k outwards as far as possible,

checking if every single cell inside that new boundary matches the corner cell.

- **Time:** $O((M * N)^2)$.
- **Space:** $O(1)$.
- **Bottleneck:** We are repeatedly verifying the same inner cells. If we know a 2×2 square is all 1s, we shouldn't have to re-check those cells to see if a 3×3 square is valid.

- **Optimal Solution (2D Dynamic Programming):**

- **Logic:** Create two DP tables: $dp1[i][j]$ (for squares of 1s) and $dp0[i][j]$ (for squares of 0s). The value at $dp[i][j]$ represents the maximum side length of a valid square whose **bottom-right corner** is at (i, j) .
- **The Golden Rule:** For a cell $matrix[i][j]$ to be the bottom-right corner of a 3×3 square, three things must be true:
 1. The square ending exactly above it must be at least 2×2 .
 2. The square ending exactly to its left must be at least 2×2 .
 3. The square ending exactly top-left must be at least 2×2 .
- **The Formula:** $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$.
- **Time:** $O(M * N)$ (A single pass through the matrix).
- **Space:** $O(M * N)$ to store the DP tables.

- **Oral Exam "Gotchas":**

- **Space Optimization Flex:** If the examiner asks if $O(M * N)$ space is strictly required, tell them **NO**. Since row i in the DP table only ever looks at row $i-1$ (the row directly above it), we can optimize the space to $O(N)$ by just keeping a 1D array representing the "previous row" and updating it as we move down!
- **The "Or" Requirement:** The standard version of this problem (like on LeetCode) only asks for squares of 1s. This specific problem asks for 0s *or* 1s. Ensure you mention that you must track both simultaneously and take the overall maximum.
- **Out of Bounds Errors:** To prevent crashing when checking $i-1$ and $j-1$ for the first row and column, you should create the DP table with size $(M+1) \times (N+1)$ and leave the 0th row/col as zeros.

Problem 11:

Consider k data clusters, each of which can be assigned to one of 2 groups: Group 1 or Group 2. Cluster i has weights (a_i, b_i) : if cluster i is assigned to Group 1 the weight gained is a_i ; otherwise the weight is b_i .

We need to select n clusters into Group 1 and m clusters into Group 2, with each cluster assigned to at most one group, so into $n + m \leq k$.

Task: Find an assignment of clusters to the two groups such that the total weight of all selected clusters is maximized.

Input:

- First line: k, n, m ($1 \leq k \leq 10^3; n + m \leq k$)
- Line i of the k following lines: a_i, b_i ($1 \leq a_i, b_i \leq 10^5$)

Output:

- A single integer — the maximum total weight.

INPUT	OUTPUT	Explanation
4 2 1 4 9 3 5 7 2 5 5	21	4 9 3 5 7 2 5 5

Problem 11: Cluster Assignment (Group 1 vs Group 2)

Pattern: Greedy Sorting + Prefix/Suffix Priority Queues

Constraints: $k \leq 10^3, n+m \leq k$.

- **Baseline (3D Dynamic Programming):**

- **Logic:** Process clusters one by one. For each cluster, you have 3 choices: assign to G1, assign to G2, or skip it. Keep a DP table tracking `(current_cluster, count_G1, count_G2)`.
- **Time:** $O(k * n * m)$.
- **Space:** $O(n * m)$ if you optimize space by only keeping the previous row.
- **Bottleneck:** While this DP will actually pass the time limit for $k=1000$, it is unnecessarily slow. You are testing millions of invalid combinations.
- **Optimal Solution (The "A-Grade" Mathematical Trick):**
 - **Logic:**
 - **The Sort:** Calculate the difference `(a - b)` for every cluster. Sort all clusters by this difference in descending order.
 - **The Split:** Because of this sorting, *every* cluster that should belong to Group 1 will naturally fall to the left of *every* cluster that should belong to Group 2! We just don't know exactly where the dividing line is.
 - **Prefix/Suffix Heaps:** Create an array `prefix_a` where `prefix_a[i]` stores the sum of the top `n` values of `a` from the left side. Create an array `suffix_b` where `suffix_b[i]` stores the sum of the top `m` values of `b` from the right side. We use Min-Heaps to easily keep track of the top elements.
 - **Find the Best Cut:** Loop a dividing line `i` through the array. The answer is simply the maximum of `prefix_a[i-1] + suffix_b[i]` for all valid split points.
 - **Time:** $O(k \log k)$. Sorting takes $O(k \log k)$. The priority queues process `k` elements in $O(k \log n)$ and $O(k \log m)$. Overall Time: $O(k \log k)$.
 - **Space:** $O(k)$ for the prefix/suffix arrays and heaps.
- **Oral Exam "Gotchas":**
 - **Why does sorting by `(a - b)` work?** This is the most important question the lecturer will ask!
 - **Your Defense:** "Suppose we randomly picked an item on the left for Group 2, and an item on the right for Group 1.

Because the left item has a higher $(a - b)$ value than the right item, swapping their groups mathematically guarantees that the total sum will either increase or stay exactly the same. Therefore, the optimal assignment **MUST** have all Group 1 items to the left of all Group 2 items in this sorted order."

- **Valid Split Range:** You cannot just put the dividing line anywhere. To pick n items for G1 and m items for G2, the left side must have *at least* n clusters, and the right side must have *at least* m clusters. The loop for the dividing line must strictly go from n to $k - m$.

Problem 12:

Construct a square with integer area equal to a given positive integer S using integer coordinates in the range from -10^9 to 10^9 .

Input: A positive integer $S(S \leq 10^3)$.

Output: The 4 vertices of the square listed in clockwise or counterclockwise order. If no solution exists, output Impossible.

INPUT	OUTPUT
5	1 2 2 4 4 3 3 1
3	Impossible

1. The Math

A square's area is the square of its side length. In a coordinate system, the length of a segment from $(0,0)$ to (a,b) is $\sqrt{a^2 + b^2}$. Thus:

$$\text{Area} = a^2 + b^2 = S$$

2. The Vertices

Once you find a and b (using a simple loop), you can construct the square starting from the origin $(0,0)$. To ensure the sides are perpendicular and equal in length, you use the property that the vector (a,b) is perpendicular to $(-b,a)$.

The four vertices are:

- $P_1 = (0, 0)$
- $P_2 = (a, b)$
- $P_3 = (a - b, b + a)$ (This is $P_2 +$ the perpendicular vector $(-b, a)$)
- $P_4 = (-b, a)$

3. Logic Summary

1. **Iterate** through possible values of a from 0 to $\sqrt{S}$.
2. **Check** if $S - a^2$ is a perfect square (b^2).
3. **Output** the four points derived from a and b if found.
4. **Output** "Impossible" if no such a and b exist.

Time: $O(\sqrt{S})$

Space: $O(1)$.

Problem 13:

Given $n \left(n \leq 10^3 \right)$ points with integer coordinates such that no two coincide and no three are collinear (n is even).

Task: Select 2 of the n points such that the line through them splits the remaining points into two equal halves.

Input:

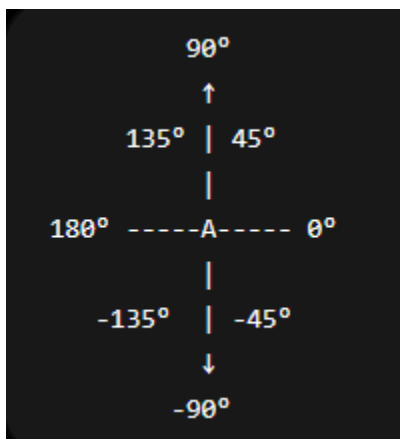
- First line: $n(2 \leq n \leq 10^3)$
- Line i of the n following lines: $x_i, y_i (|x_i|, |y_i| \leq 10^6)$ — coordinates of point i .

Output:

- Two integers — the indices of the two selected points.

Example:

INPUT	OUTPUT
6 3 5 1 3 3 1 6 1 8 3 6 5	2 5



Fix a point A.

Compute the polar angle of every other point around A.

Sort the points by angle.

Duplicate the sorted array to handle circular wraparound.

Use two pointers:

l chooses the line A -> point[l]

r expands while points stay on the left side of the line

The interval (l+1 ... r-1) contains all points in the open semicircle to the left of the line.

If the number of points there equals $(n-2)/2$, then the line splits the remaining points equally, so output the pair.

Why duplication helps:

Angles are circular.

A valid semicircle may wrap from the end of the array back to the beginning.

Duplicating converts the circular order into a linear interval so sliding window works normally.

Cross product:

$\text{cross}(A,B,C) > 0$:

C is on the left side of directed line A -> B.

Time Complexity:

Sorting for each point: $O(n \log n)$

Two-pointer sweep for each point: $O(n)$

Repeated for all n points:

$O(n^2 \log n)$

Space Complexity:

$O(n)$

Problem 14:

Given a convex polygon with n vertices with integer coordinates x_i, y_i . The vertices are listed counterclockwise and no three are collinear. Find a diagonal connecting 2 vertices that splits the polygon into 2 parts with minimum area difference.

Input

- First line: $n (n \leq 10^3)$
- Line i of the n following lines: $x_i, y_i (|x_i|, |y_i| \leq 10^9)$.

Output:

- Two integers — the indices of the two chosen vertices.

Example:

INPUT	OUTPUT
4 0 2 0 0 2 0 3 3	2 4

Idea:

- Let the total polygon area be S .
- A diagonal (i, j) splits the polygon into 2 parts with areas A and $S - A$.

- The area difference is: $|A - (S - A)| = |2A - S|$
- So we want one subpolygon area to be as close as possible to $S/2$.

Key Observations:

- The polygon is convex and vertices are given counterclockwise.
- For a fixed vertex i , as j moves forward, the subpolygon area increases monotonically.
- Therefore: $|2A - S|$ first decreases, reaches a minimum, then increases.
- This allows the use of the two-pointer (rotating calipers) technique.

How the algorithm works:

- Compute the total polygon area using the shoelace formula.
- Fix a left pointer i .
- Move right pointer j forward while the split becomes more balanced.
- The area is updated incrementally by adding/removing triangle areas instead of recomputing from scratch.
- Since both pointers only move forward, the total complexity is linear.

Why it is correct:

- In a convex polygon, increasing j always increases the subpolygon area.
- Once moving j no longer improves the balance, further movement will only worsen it.
- Thus the optimal j for each i is found greedily.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Problem 15:

Given two circles on a plane with equal radius R and centers at (x_1, y_1) and (x_2, y_2) . Compute the area of the union of the two circles. All center coordinates and the radius are integers.

Input:

- Contains integers x_1, y_1, x_2, y_2, R ($1 \leq x_1, y_1, x_2, y_2, R \leq 100$)

Output:

- A single real number — the area of the union, to exactly 3 decimal places.

Example:

INPUT	OUTPUT
1 2 3 4 2	22.850

Intersection Area:

$$A_{\text{int}} = 2R^2 \arccos\left(\frac{d}{2R}\right) - \frac{d}{2} \sqrt{4R^2 - d^2}.$$

Problem 16:

Given a tree with n vertices. The distance between two vertices u, v is the number of edges on the path from u to v . The width of the tree is the sum of distances over all pairs $u, v (u < v)$. Determine the width of the tree.

Input

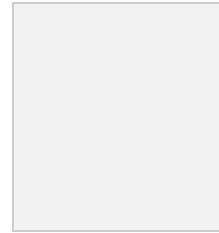
- First line: $n (n \leq 10^3)$.
- $n - 1$ following lines: u_i, v_i — an edge of the tree.

Output:

- A single integer — the width of the tree.

Example:

INPUT	OUTPUT
5 1 2 1 3 3 4 3 5	18



Idea:

Instead of calculating distances for every pair of vertices directly, compute the contribution of each edge.

If an edge is removed, the tree splits into 2 components:

one side has size s

the other side has size $n - s$

Any pair of vertices chosen from different sides must use this edge in their path. Therefore, this edge contributes:

$$s \times (n - s)$$

to the total width.

Root the tree at any node and use DFS to compute subtree sizes.

For each edge (parent $\rightarrow$ child):

$$s = \text{subtree_size}[\text{child}]$$

$$\text{contribution} = s \times (n - s)$$

Sum all edge contributions.

Time Complexity: $O(n)$

Explanation:

- DFS visits each node and edge once
- contribution of each edge is computed once

Space Complexity: $O(n)$

Explanation:

- adjacency list stores $O(n)$ edges
- recursion stack and subtree array use $O(n)$ space

Problem 17:

Given a positive integer n . Find the smallest positive integer m with at most 9 digits satisfying

- m it contains only the digits $\{0, 1\}$
- m it is a multiple of n .

Input:

- A positive integer $n (n \leq 100)$

Output:

- A single integer — m

Example:

INPUT	OUTPUT
6	1110

Use BFS to build numbers consisting only of digits 0 and 1.

Each BFS state represents a remainder modulo n .

Start from number 1 with remainder $1 \% n$.

From a current number, generate new numbers by appending:

- 0

- 1

Compute the new remainder using:

- $(\text{rem} * 10 + \text{digit}) \% n$

Use a **visited** array to avoid processing the same remainder multiple times.

Since BFS explores level by level, the first valid multiple found is the smallest one.

Ignore numbers exceeding 9 digits.

Time Complexity: $O(n)$

- There are at most n different remainders, and each is processed once.

Space Complexity: $O(n)$

- The queue and visited array store at most n states.

Problem 18:

Given a sequence $a_1, a_2, \dots, a_n$ which is a permutation of integers from 1 to n .

The following operation is allowed: choose a position $i (2 \leq i \leq n)$ and reverse the prefix $a_1, a_2, \dots, a_i$, leaving the rest unchanged.

Task: Find the minimum number of operations to sort the sequence in non-decreasing order.

Input:

- First line: $n (1 \leq n \leq 8)$
- Second line: $a_1, a_2, \dots, a_n$

Output:

- A single integer — the minimum number of operations.

Example:

INPUT	OUTPUT	Explanation
5 5 2 3 4 1	4	5 2 3 <u>4</u> 1 4 3 <u>2</u> 5 1 2 3 4 <u>5</u> 1 5 4 3 2 <u>1</u> 1 2 3 4 5

Idea:

- From initial state, use BFS to consider all possible next state.
- Check for the current state to match the target state.

TimeO($n \cdot n!$) — instant for $n \leq 8$

SpaceO($n \cdot n!$) for the queue + visited map

Problem 19:

Given n consecutive segments; segment i has height h_i . The height difference between any two consecutive segments is at most 1. We want to excavate as deep as possible and plan to spend T coins. Reducing a segment's height by 1 unit costs 1 coin.

Task: Given the heights $h_1, h_2, \dots, h_n$ and budget T . Determine the minimum achievable height (deepest excavation) using at most T coins, while keeping consecutive height differences at most 1.

- The first line contains two positive integers n, T ($n \leq 10^5, T \leq 10^{18}$)

- The second line contains n non-negative integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$)

Output

A single integer — the minimum achievable height.

Example:

INPUT	OUTPUT	Explanation															
4 3 1 1 1 1	-1	<table><tr><td>1</td><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td></td><td></td><td></td><td></td></tr><tr><td>-1</td><td></td><td></td><td></td><td></td></tr></table>	1					0					-1				
1																	
0																	
-1																	
4 3 1 2 2 1	0	<table><tr><td>2</td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td></td><td></td><td></td><td></td></tr></table>	2					1					0				
2																	
1																	
0																	

Idea:

- We want the minimum possible height after excavation.
- Since adjacent height differences must remain at most 1, if the minimum height is H at position i , then every position j can have height at most:

$$H + |i - j|$$

- This forms a “tent” shape centered at i .
- If $a[j]$ exceeds this limit, we must excavate the excess:

$$a[j] - (H + |i - j|)$$

- The total required excavation cost is:

$$\sum \max(0, a[j] - (H + |i - j|))$$

- If this cost is $\leq T$ for some center i , then height H is achievable.

Algorithm:

1. Binary search the answer H .
2. For each candidate H :
 - Try every position i as the location of the minimum height.
 - For every position j :
 - Compute the maximum allowed height:

$$H + |i - j|$$

- If $a[j]$ is larger, add the required excavation cost.
 - If any center i gives total cost $\leq T$, then H is feasible.
3. If H is feasible:
 - Try smaller heights.
 4. Otherwise:
 - Try larger heights.
 5. Output the minimum feasible H .

Time Complexity:

- $\text{check}(H)$:
 - Try all centers: $O(n)$
 - Scan all positions: $O(n)$
 - Total: $O(n^2)$
- Binary search performs $O(\log A)$ checks, where A is the height range.

Overall:

- $O(n^2 \log A)$

Space Complexity:

- $O(n)$
- Stores the input array only.

Problem 20:

Consider all permutations of $(1, 2, \dots, n)$. Assume they are sorted lexicographically and numbered from 1 to $n!$

Example for $n = 3$: there are 6 permutations: $(1,2,3)$; $(1,3,2)$; $(2,1,3)$; $(2,3,1)$; $(3,1,2)$; $(3,2,1)$

Task: Given a permutation $(p_1, p_2, \dots, p_n)$ find its rank x ; and conversely: given a rank y ($1 \leq y \leq n!$) find the permutation $(q_1, q_2, \dots, q_n)$ with rank y .

Input:

- Line 1: Contains n numbers $p_1, p_2, \dots, p_n$ ($n \leq 20$)
- Line 2: Contains number y

Output:

- Line 1: Write x
- Line 2: Write n numbers $q_1, q_2, \dots, q_n$

Example:

INPUT	OUTPUT
2 1 3	3
4	2 3 1

Idea

- Use the factorial number system (Lehmer code).
- For ranking:
 - At each position, count how many unused numbers smaller than the current value exist.
 - Each contributes **count $\times$ (remaining-1)!** permutations.
- For unranking:
 - Convert **(y-1)** into factorial representation.
 - Repeatedly choose the corresponding unused number.

Time Complexity

- Ranking: $O(n^2)$
- Unranking: $O(n^2)$

($n \leq 20$, so this is fast enough.)

Space Complexity

- $O(n)$

We want to find how many permutations come before a given permutation ppp in lexicographic order.

Suppose:

$$p = (p_1, p_2, \dots, p_n) \quad p = (p_1, p_2, \dots, p_n)$$

The main idea is:

- We compare permutations from left to right.
 - A permutation becomes smaller as soon as it has a smaller number at the first position where it differs.
-

Core Idea

At position i :

- positions $1 \dots i-1$ are already fixed and identical to ppp ,
- now we look at position i .

We ask:

How many permutations would come before ppp if we put a smaller unused number here?

That is what we are “choosing”.

Problem 21:

Output the country SSS, there are N cities and M railway routes connecting them. All route lengths are distinct and it is guaranteed that every pair of cities is connected (directly or indirectly).

The government will upgrade a set of routes to a new high-speed rail system so that every pair of cities remains connected through upgraded routes. The goal is an optimal plan that minimizes the total length of upgraded routes (i.e., a Minimum Spanning Tree).

To handle contingencies, the government also wants to identify K distinct spanning-tree plans. Two plans differ if at least one route appears in one but not the other. Find the K -th backup plan for safety and reliability.

Output:

- The first line contains 3 integers N , M , K
- The next M lines each contain 3 integers u , v , c (edge from u to v with length c)

Output:

- Output a single integer — the total route length of the K -th spanning-tree plan. K .

Example:

INPUT	OUTPUT	Explanation
5 5 2 1 2 2 1 3 4 2 4 3 2 5 1 3 5 5	11	Routes in plan K include: 1 2 2 5 2 4 3 5

- Sort all edges by weight and compute the normal MST using Kruskal.
- Use a priority queue to enumerate spanning trees in increasing total cost order.
- Each state represents a subset of spanning trees defined by:
 - forced edges (must appear),

- banned edges (cannot appear).
- For each state, run a constrained Kruskal algorithm:
 - first add all forced edges,
 - then run normal Kruskal while skipping banned edges.
- The resulting tree is the minimum spanning tree inside that state/region.
- Suppose the current spanning tree contains edges:
 - $e_1, e_2, e_3, \dots, e_{n-1}$
- Generate child states:
 - forbid e_1
 - force e_1 , forbid e_2
 - force e_1, e_2 , forbid e_3
 - ...
- This partitions all remaining spanning trees into disjoint regions:
 - every spanning tree belongs to exactly one region,
 - no duplicates occur.
- Push the minimum tree of every valid region into the priority queue.
- Repeatedly pop the smallest-cost state from the heap.
- The K -th popped spanning tree is the K -th minimum spanning tree.

Algorithms used:

- Kruskal Minimum Spanning Tree
- Disjoint Set Union (DSU / Union-Find)
- Best-first search using priority queue
- State partitioning / K -best enumeration

Time Complexity:

- One constrained MST build:
 - $O(M \alpha(N))$
- Each extracted state may generate up to $N-1$ child states.
- For K spanning trees:
 - $O(K \cdot N \cdot M \cdot \alpha(N))$
- Often simplified as:
 - $O(KNM)$

Space Complexity:

- DSU and graph storage:
 - $O(N + M)$
- Priority queue may store up to $O(KN)$ states in worst case.
- Each state stores edge constraints.
- Total:
 - $O(KN + M)$

Problem 22:

In a mystical land, there is a kingdom called Graphonia with N cities and M roads connecting them. The King of Graphonia wants to optimize the road network. He wants to know the number of vertex pairs that become disconnected if each road is removed one at a time.

Input:

- The first line contains 2 integers N, M
- M The next M lines each contain 2 integers $u\ v$ (edge from u to v)

Output:

- Consists of M lines, each containing the number of vertex pairs disconnected when that edge is removed

Example:

INPUT	OUTPUT
5 5	0
1 2	0
2 3	4 (vertex 4 cannot reach vertices
3 4	1, 2, 3, 5)
3 5	0
5 1	0

Idea

- Use Tarjan DFS algorithm to find bridges.
- For every DFS tree edge (u, v) :
 - if $low[v] > tin[u]$, then (u, v) is a bridge.
- $subtree[v]$ stores the number of vertices in v 's DFS subtree.
- Removing a bridge splits the graph into:
 - one component of size $subtree[v]$
 - another of size $n - subtree[v]$
- Therefore the number of disconnected vertex pairs is:

$$subtree[v] \times (n - subtree[v]) + subtree[v] \times (n - subtree[v])$$

- Non-bridge edges do not disconnect any vertices, so their answer is 0.

Important Definitions

- $tin[u]$:
discovery time of vertex u
- $low[u]$:
smallest discovery time reachable from the subtree of u
using tree edges and at most one back edge

Bridge Condition

For DFS tree edge (u, v) :

$$low[v] > tin[u]$$

means the subtree of v cannot reach u or any ancestor of u ,
so (u, v) is a bridge.

Time Complexity

$$O(N+M)$$

because DFS visits each vertex and edge once.

Space Complexity

$O(N+M)$

for adjacency list and DFS arrays.